

Project Executable Files

Date	04 July 2026
Team ID	ramanharshitha2005@gmail.com
Project Name	OptiCrop - Smart Agricultural Production Optimization Engine
Maximum Marks	3 Marks

Project Executable Files

This section requires submission of all executable and deployable files for your project. The evaluator must be able to run or deploy your solution using the provided files and instructions.

Step 1: Submission Checklist

S.No	Item to Submit	Submitted (Yes / No / NA)
1	Complete source code (all files and folders)	Yes
2	README / Setup Guide (instructions to run the project)	Yes — README.md in repo root
3	requirements.txt / package.json / dependency file	Yes — requirements.txt in repo root
4	Database schema / seed files (if applicable)	Yes — data/Crop_recommendation.csv + data/opticrop-data.json (flat-file, no DB schema needed)
5	Environment configuration file (.env.example or similar)	NA — No API key or .env needed. Open-Meteo requires no key.
6	Deployed application URL (if hosted)	NA — Local only. Run: python3 server.py → http://127.0.0.1:8000
7	APK / Executable binary (if applicable for mobile/desktop apps)	NA — Web application only. No mobile APK.
8	Dockerfile / Containerization config (if applicable)	NA — Not containerised. pip install + python3 server.py is sufficient.
9	Test files and test results	Yes — Manual test results documented in Performance Testing template. models/confusion_matrix.png and models/disease_confusion_matrix.png generated during training.
10	Demo video or walkthrough (if required)	Yes — Screen recording demonstrating all 5 pages and all 7 API features.

Step 2: File / Folder Structure

Project directory structure (Smart_Bridge-master/):

File / Path	Description
server.py	Main Python HTTP server — all 7 API routes + static file serving. Entry point to run the app.
requirements.txt	All Python dependencies with minimum version pins.

README.md	Full project description, dataset setup guide, run instructions, and project structure overview.
data/Crop_recommendation.csv	Kaggle crop dataset — 2200 rows, 8 columns (N, P, K, temperature, humidity, ph, rainfall, label), 22 crop classes.
data/opticrop-data.json	Fallback crop profiles + disease records used when CSV is unavailable and for disease model training.
models/crop_model.joblib	Trained ensemble bundle: RF (n=350), DT, SVC + LabelEncoder + feature list. Generated by ml/train_model.py.
models/disease_model.joblib	Trained RF disease classifier + encoders. Generated by ml/train_disease_model.py.
models/image_disease_model.joblib	Pixel-feature RF for image disease (lightweight fallback). Generated by ml/train_image_model.py.
models/confusion_matrix.png	Crop prediction confusion matrix heatmap (Seaborn). Generated during training.
ml/train_model.py	Trains crop recommendation ensemble (RF+DT+SVC) and saves crop_model.joblib.
ml/train_disease_model.py	Trains disease RF classifier with data augmentation and saves disease_model.joblib.
ml/train_image_model.py	Trains pixel-feature RF for image disease classification.
ml/train_image_model_kaggle.py	Trains Keras CNN on PlantVillage dataset and saves .h5 model (optional, requires dataset).
ml/predict.py	Standalone crop prediction script for local testing without running the server.
ml/validate_kaggle_dataset.py	Validates the PlantVillage dataset structure before CNN training.
web/index.html	Home page — hero section, feature cards, project description.
web/recommend.html	Crop Recommendation + Suitability Assessment page (Scenario 1 & 2).
web/disease.html	Plant Disease Detection page — symptom form + image upload.
web/insights.html	Research Analytics Dashboard — Chart.js charts, filters, CSV export (Scenario 3).
web/contact.html	Contact page.
web/styles.css	Shared responsive CSS stylesheet for all 5 pages.
web/script.js	Shared JavaScript for API calls, result rendering, GPS weather, CSV export.
web/assets/	PNG hero images for each page (farmer_hero.png, disease_hero.png, insights_hero.png, contact_hero.png, opticrop_icon.png).
scripts/setup_kaggle.py	Helper script to download Kaggle datasets programmatically.
scripts/setup_kaggle.sh	Shell script alternative for Kaggle dataset setup.

Step 3: Deployment / Access Details

Hosted / Deployed URL	http://127.0.0.1:8000 (local only — no external hosting in current version)
Login Credentials (Demo)	No login required. Open the URL directly in any browser after running server.py.
Platform / Hosting Provider	Local machine. Python built-in http.server — no external web server needed.
Repository Link	https://github.com/RamanHarshitha/Smart_Bridge
Demo Video Link	https://drive.google.com/file/d/1kM7wFXCwfMNyYvx6zQWZNbAnXC8tAqwq/view?usp=drivesdk

Step 4: Run Instructions

Step 1 — Clone the repository	git clone https://github.com/RamanHarshitha/Smart_Bridge cd Smart_Bridge-master
Step 2 — Install dependencies	pip install -r requirements.txt
Step 3 — Add dataset (required for full functionality)	Download Crop_recommendation.csv from Kaggle and place it in data/Crop_recommendation.csv
Step 4 — Train the crop model	python3 ml/train_model.py (Generates models/crop_model.joblib)
Step 5 — Train the disease model	python3 ml/train_disease_model.py (Generates models/disease_model.joblib)
Step 6 — Run the server	python3 server.py (Server starts at http://127.0.0.1:8000)
Step 7 — Open in browser	Navigate to http://127.0.0.1:8000 and explore all features.

Step 5: Known Issues / Limitations

S.No	Known Issue / Limitation	Workaround / Status
1	Image disease model requires PlantVillage dataset (~5 GB) to train the full Keras CNN. Without it, the lightweight pixel-feature RF model is used as fallback.	Pixel-feature RF model (image_disease_model.joblib) is automatically used as fallback. Accuracy is lower but functional.
2	Open-Meteo weather API requires internet connectivity. If offline, the weather auto-fill will fail.	User can manually enter temperature, humidity, and rainfall values in the form without using the weather auto-fill.
3	No user authentication — the app is open to all users on the local network (0.0.0.0:8000).	Intended design for local/LAN use. Bind to 127.0.0.1 instead of 0.0.0.0 in server.py to restrict to localhost only.